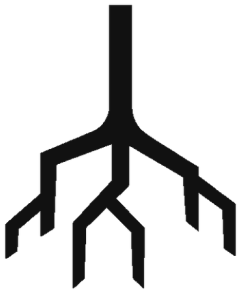

# dstack Security Review

## rootward

Static security review of a Web3 protocol built on a cloud TEE

commit 174d85819a94

19 August 2026

# Contents

# 1. About rootward

rootward is a static auditor for Web3 protocols built on cloud trusted execution environments: AWS Nitro Enclaves, dstack, EigenCompute, and Google Cloud Confidential Space. It runs a catalog of rules derived from the Bluethroat Labs TEE Security Handbook against a repository, reading source, container and deployment configuration, KMS key policy, OS and firmware build recipes, and the built enclave image.

Every rule cites the handbook section it comes from and declares the conditions under which it is wrong. Detection is measured rather than asserted: recall is established by injecting each catalogued defect into a clean tree and asking whether that rule fires.

# 2. Disclaimer

A static review can never establish the absence of vulnerabilities. This one reads a repository at a single commit. It does not observe a running enclave, does not fetch an attestation document from live hardware, does not read the KMS policy actually attached to the deployed key, and does not reason about the economics or the key ceremony of the protocol under review.

Findings are produced by pattern and format analysis and each one should be read against its cited evidence before it is acted on. Section 9 lists what this review structurally could not check, and a clean result above does not extend to anything in that list. A manual review by a qualified engineer remains necessary.

# 3. Introduction

A static security review of **dstack** was performed with rootward, focused on the properties a cloud TEE deployment depends on: attestation verification, measurement pinning, the trust boundary against the parent instance, secret handling, and the configuration of the image the workload runs in.

Review commit: `174d85819a94cbc3af111a7cd7bb526bbd57c7e9`

# 4. About the review target

Platform detected: **nitro, dstack, confidential-space**. Platform detection decides which rules apply, in both directions, so a report for one platform does not carry another platform's inapplicable findings.

- dstack reference `CODE_OF_CONDUCT.md:63, REUSE.toml:2`
- Phala cloud `CODE_OF_CONDUCT.md:63, REUSE.toml:3`
- dstack tooling `REUSE.toml:236, CHANGELOG.md:155`
- tappd socket `CHANGELOG.md:349, dstack/run-tests.sh:14`
- governance contracts `CHANGELOG.md:151`
- RTMR `CHANGELOG.md:30, CONTRIBUTING.md:65`
- app-compose manifest `CHANGELOG.md:21, CLAUDE.md:209`
- TDX guest device `CONTRIBUTING.md:64, docs/native-tee-interfaces.md:14`

# 5. Risk classification

| Severity | Impact: High | Impact: Medium | Impact: Low |
|---|---|---|---|
| Likelihood: High | Critical | High | Medium |
| Likelihood: Medium | High | Medium | Low |
| Likelihood: Low | Medium | Low | Low |

## 5.1. Impact

- **High**: The enclave's core guarantee fails. Secrets leave it, or an unmeasured workload is accepted as a measured one.
- **Medium**: A control that other controls depend on is weakened, without directly exposing key material.
- **Low**: Unexpected behaviour with no direct path to key material or to accepting an unmeasured workload.

## 5.2. Likelihood

- **High**: The defect is present as written and reachable on the deployed path. No additional conditions are required for it to matter.
- **Medium**: The defect is present and a plausible benign reading of the same code exists. Confirmation against the cited line is required.
- **Low**: Reported for completeness. The pattern is indicative rather than conclusive and needs adjudication by someone who knows the deployment.

## 5.3. Action required for severity levels

- **Critical**: Must fix before the enclave handles anything of value.
- **High**: Must fix. The guarantee the deployment is sold on does not hold without it.
- **Medium**: Should fix. Weakens a control that other controls are assumed to rest on.
- **Low**: Could fix. Reachable only under conditions that are unlikely or costly.

# 6. Security assessment summary

Review commit hash: `174d85819a94cbc3af111a7cd7bb526bbd57c7e9`

## Scope

The following were in the scope of the review:

- Application source across Rust, Go, Python, JavaScript and TypeScript
- Container and deployment configuration
- KMS key policy
- OS and firmware build recipes (BitBake, EDK II)
- The built enclave image, where one is present in the tree

Excluded: paths that are test data, mocks, simulators, fixtures or vendored samples, and anything that is not resident in the repository at the commit above. Live infrastructure was not touched: no credentials were used, no attestation was fetched from running hardware, and the deployed KMS policy was not read.

## Analysis performed

- **Deterministic rules**: parse, AST and configuration checks over source, containers, deployment manifests and KMS policy.
- **Code-pattern rules**: a TEE-specific semgrep ruleset across Rust, Go, Python, JavaScript and TypeScript.
- **OS and firmware**: BitBake recipes and EDK II build invocations, which decide what the machine the workload runs on was built from.
- **Image analysis**: where an enclave image is present: format parse, ramdisk walk, measurement recomputation and a secret scan of the contents.

## Attack vectors covered

Each of the following was looked for across the scope above and no instance was found. They are listed because a findings list on its own does not distinguish what was checked and held from what was never checked.

### 1. Enclave launched in debug mode, voiding cryptographic attestation

#### Description

A debug-mode enclave produces attestation documents whose PCRs are entirely zeros, and the handbook states plainly that these cannot be used for cryptographic attestation. Debug mode also exposes the enclave console to the parent. A deployment script carrying --debug-mode therefore turns off the single guarantee the whole architecture rests on, silently, with no error at runtime.

#### Protection

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CFG01-debug-mode-launch.

### 2. Verifier accepts an all-zero PCR map

#### Description

The counterpart to BT-CFG01, on the verifying side. Debug enclaves emit all-zero PCRs, so a verifier that does not explicitly reject them will accept any debug enclave as genuine. This is the failure that converts a deployment mistake into an exploitable one, and it is invisible in testing because a debug enclave is exactly what a developer runs locally.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CFG02-zero-pcr-accepted.

### 3. PCR value pinned in KMS policy does not match the PCR recomputed from the EIF

**Description**

The strongest check in the catalog, because both sides are computable and the output is two concrete hashes rather than a judgment. If the PCR0 recomputed from the committed EIF build does not equal the value pinned in the KMS key policy, then either the policy authorises an image nobody can account for, or the deployed image cannot obtain its keys. Both are defects, and which one it is tells you a great deal about the deployment.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CFG03-pcr-policy-eif-mismatch.

### 4. No key rotation path, so a single compromise is permanent

**Description**

The handbook lists absent key rotation as a residual risk at Layer 2, phrased as leading to eventual compromise. The 2025 physical attacks make the reasoning concrete: WireTap, Battering RAM, and TEE.fail demonstrated that memory-bus interposition against SGX, TDX, and SEV-SNP is achievable with roughly a thousand dollars of hardware. A design whose keys never change assumes the hardware guarantee holds forever, and that assumption now has published counterexamples.

**Protection**

No instance was found in the reviewed scope. The check is hybrid and is recorded under BT-CFG05-no-key-rotation.

### 5. Confidential Space attestation token decoded but never verified

**Description**

A Confidential Space attestation is a JWT signed by Google's attestation verifier. Its payload is base64, not ciphertext, so decoding it is a formatting operation that any party can perform on any string of the right shape. The security property comes entirely from checking the signature against Google's published JWKS. Code that splits the token on '.', base64-decodes the middle segment and reads claims out of it obtains attacker-controlled data wearing the costume of a hardware attestation, and reads as verification in review because the variable is called a token and the fields are called claims.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CS01-attestation-token-unverified.

### 6. Attestation verified but the platform claims are never compared

**Description**

A valid signature proves Google issued the token. It does not prove the workload runs on Intel TDX, that secure boot was enabled, that the platform TCB is current, or that the token was minted for this audience rather than replayed from another service. Each of those is a separate claim - hwmodel, swname, secboot, tcbstatus, aud, iss - and each has to be compared against an expected value. A correctly signed token from a debug VM with secure boot off satisfies the signature check and asserts, in plain text, that it is a debug VM with secure boot off.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CS02-platform-claims-unchecked.

## 7. Attestation failure is caught and the workload starts anyway

**Description**

An enclave that cannot prove what it is running is indistinguishable from one running something else. A catch block that logs a warning and continues therefore starts the workload - and releases whatever the workload holds - in precisely the case the attestation existed to rule out. The pattern is common because it is the behaviour that keeps a service up during a platform outage, and the cost of that availability is that the security control becomes advisory.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CS03-attestation-fail-open.

## 8. Project presents attestation with no hardware root of trust

**Description**

A signature proves that whoever holds the signing key signed the bytes. Attestation proves that the code which signed them is the code you published, running on hardware that vouches for it. Projects conflate the two: they sign every response with a key the platform provided, emit headers naming it an attestation, ship a verification library, and use the word throughout their documentation, while performing no quote fetch, no token verification, and no certificate-chain walk anywhere in the tree. Every individual piece works, so review finds nothing wrong; the gap is that the security property being advertised was never implemented at all. A consumer who checks the signature and sees it pass will believe a great deal more than was proven.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-CS04-attestation-without-hardware-root.

## 9. dstack-KMS configured for simple duplication rather than threshold sharing

**Description**

dstack-KMS offers two root-key strategies. Simple duplication has one node generate the root key and share it with peers after verifying attestation, which is highly available and leaves a single point of total compromise. Shamir-based MPC splits the root key so that reconstruction requires a threshold of nodes to collaborate. Choosing duplication means one compromised node yields every derived key in the system, which is the exact failure Layer 4 exists to prevent.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-DS03-kms-simple-duplication.

## 10. Measured boot values not compared against an expected policy

**Description**

dstack's verification chain is measured boot from hardware through OS to containers, compared against expected policy. Collecting the measurements without comparing them is the dstack analogue of BT-T01: the quote verifies, and it attests to nothing in particular.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-DS04-no-rtmr-policy-check.

## 11. Zero Trust TLS domain binding not established for the workload

**Description**

dstack-Gateway provides a cryptographic binding between a domain and a verified workload, so a client reaching the domain can confirm which code answers, without trusting a certificate authority. A deployment that terminates TLS conventionally keeps the CA in the trust base and loses the ability to prove that the responder is the attested workload rather than a proxy in front of it.

**Protection**

No instance was found in the reviewed scope. The check is hybrid and is recorded under BT-DS05-gateway-no-domain-binding.

## 12. Security check disables itself or degrades to a weaker one

**Description**

Two shapes, one failure. A verification function that returns successfully when its expected value is unset turns a missing pin into a pass, so a deployment that forgot to configure the pin is indistinguishable from one that satisfied it. A signer whose catch block returns a plain hash produces something well-formed that takes no key, so every consumer accepts forged input. Both are silent: the degraded path returns the same shape as the healthy one, and no log line distinguishes them.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-EC04-check-silently-disabled.

## 13. Unauthenticated HTTP route enumerates the process environment

**Description**

On EigenCompute the environment is where KMS puts the application's secrets after attestation, so an endpoint that iterates it publishes exactly the material the enclave exists to protect. Debug routes of this kind are typically written to diagnose a deployment problem, filtered by a prefix such as KMS or TEE that selects for the most sensitive variables, and then left in place. Truncating the values is not a mitigation when they are the inputs to a key derivation.

**Protection**

No instance was found in the reviewed scope. The check is deterministic and is recorded under BT-EC06-unauthenticated-env-dump.

### 14. Effective security layer versus claimed security layer

**Description**

Not a defect rule. Produces the report headline by placing the codebase on the handbook's Layer 0 to 6 ladder and diffing that against what the project claims in its README and documentation. Each layer has prerequisites expressed as layer_required across this catalog, so the effective layer is the highest one whose prerequisites all pass. The handbook is explicit that Layer 1, TEE with no attestation verification, must not be used in production, which makes the gap between claimed and effective layer the single most decision-relevant output of an audit.

**Protection**

No instance was found in the reviewed scope. The check is semantic and is recorded under BT-LYR01-layer-scorecard.

A further 12 vectors were checked and produced no finding.

## Security layer scorecard

Effective layer **0** of a verifiable ceiling of 4. The ladder is the Bluethroat Labs layer model. Failing one rule caps the deployment below the layer that rule is required for.

| Layer | Required rules | Status |
|---|---|---|
| 1 | 15 | fails (3) |
| 2 | 19 | fails (2) |
| 3 | 3 | fails (2) |
| 4 | 2 | pass |
| 5 | 0 | unassessed (no rules) |
| 6 | 0 | unassessed (no rules) |

# 7. Executive summary

| Severity | Findings |
|---|---|
| Critical | 13 |
| High | 21 |
| Medium | 6 |
| Low | 1 |
| **Total** | **41** |

| ID | Title | Severity |
|---|---|---|
| C-01 | Decrypted data or key material reaches a log sink outside the enclave | Critical |
| C-02 | Attestation signature checked without validating the chain to the AWS Nitro root | Critical |
| C-03 | Data relayed by the parent consumed without verifying its signature | Critical |
| C-04 | Data relayed by the parent consumed without verifying its signature | Critical |
| C-05 | Attestation signature checked without validating the chain to the AWS Nitro root | Critical |
| C-06 | Data relayed by the parent consumed without verifying its signature | Critical |
| C-07 | Data relayed by the parent consumed without verifying its signature | Critical |
| C-08 | Decrypted data or key material reaches a log sink outside the enclave | Critical |
| C-09 | Decrypted data or key material reaches a log sink outside the enclave | Critical |
| C-10 | Data relayed by the parent consumed without verifying its signature | Critical |
| C-11 | Attestation signature checked without validating the chain to the AWS Nitro root | Critical |
| C-12 | Data relayed by the parent consumed without verifying its signature | Critical |
| C-13 | Certificate verification explicitly disabled on a trust-bearing connection | Critical |
| H-01 | dstack deployment without KmsAuth or AppAuth code governance | High |

| | | |
|---|---|---|
| H-02 | Container or process output stream exposed across the enclave boundary by configuration | High |
| H-03 | Container or process output stream exposed across the enclave boundary by configuration | High |
| H-04 | Host-supplied filesystem path used without rejecting symbolic links | High |
| H-05 | Container or process output stream exposed across the enclave boundary by configuration | High |
| H-06 | Non-constant-time comparison of a secret-derived value | High |
| H-07 | Host-supplied filesystem path used without rejecting symbolic links | High |
| H-08 | Host-supplied filesystem path used without rejecting symbolic links | High |
| H-09 | Non-constant-time comparison of a secret-derived value | High |
| H-10 | Container or process output stream exposed across the enclave boundary by configuration | High |
| H-11 | vsock message schema carries no nonce, counter, or signed timestamp | High |
| H-12 | Blocking vsock read with no timeout | High |
| H-13 | vsock message schema carries no nonce, counter, or signed timestamp | High |
| H-14 | Blocking vsock read with no timeout | High |
| H-15 | Panic, crash dump, or backtrace path can emit enclave memory to the parent | High |
| H-16 | vsock message schema carries no nonce, counter, or signed timestamp | High |
| H-17 | vsock message schema carries no nonce, counter, or signed timestamp | High |
| H-18 | Non-constant-time comparison of a secret-derived value | High |
| H-19 | Non-constant-time comparison of a secret-derived value | High |
| H-20 | Container or process output stream exposed across the enclave boundary by configuration | High |
| H-21 | Container or process output stream exposed across the enclave boundary by configuration | High |
| M-01 | Keys sealed to hardware rather than derived from application identity | Medium |

| | | |
|---|---|---|
| M-02 | Keys sealed to hardware rather than derived from application identity | Medium |
| M-03 | Keys sealed to hardware rather than derived from application identity | Medium |
| M-04 | Keys sealed to hardware rather than derived from application identity | Medium |
| M-05 | Keys sealed to hardware rather than derived from application identity | Medium |
| M-06 | Firmware recipe defines a secure-boot option and leaves it off | Medium |
| L-01 | Enclave image build is not reproducible, so PCR values cannot be independently confirmed | Low |

Every finding below carries the source line it was derived from. A finding is a place to look, and the cited line is what makes it checkable rather than a claim.

# 8. Findings

## 8.1. Critical findings

### [C-01] Decrypted data or key material reaches a log sink outside the enclave

`dstack/crates/qemu-acpi/scripts/differential-random.py:322`

**Description**

Secret-derived value (f"FAIL seed={args.seed:#x} case={index} config={case}") written to a log sink. Inside an enclave this reaches the operator, typically via CloudWatch. Log a keyed hash or truncated identifier.

**Evidence**

```
print( f"FAIL seed={args.seed:#x} case={index} config={case}", file=sys.stderr, )
```

**Recommendations**

Log hashed or truncated identifiers only. Where a secret-derived value must be correlated across systems, log a keyed hash of it. Route any genuinely sensitive diagnostic to an in-enclave buffer that is never flushed to the parent.

### [C-02] Attestation signature checked without validating the chain to the AWS Nitro root

`dstack/dstack-attest/src/attestation.rs:337`

**Description**

File assembles a certificate chain from the attestation document's cabundle and never verifies it against anything, the chain is built and discarded. This is the shape left behind when the verification call is removed and the plumbing around it survives. Verifying a document against the certificate embedded in that same document proves nothing: an attacker generates a keypair, signs a document asserting any PCR values they like, embeds their own certificate, and it verifies. The security of the scheme is entirely in walking the cabundle to an out-of-band pinned AWS Nitro root.

**Evidence**

```
AttestationQuote::DstackAwsNitroTpm(DstackAwsNitroTpmQuote { attestation_doc }) => {
```

**Recommendations**

Validate the full certificate chain from the leaf through the supplied cabundle to a locally pinned copy of the AWS Nitro root certificate, check validity periods, and only then trust the PCR map. Prefer an audited implementation such as evervault/attestation-doc-validation or the AWS NSM API over hand-rolling COSE verification.

### [C-03] Data relayed by the parent consumed without verifying its signature

`dstack/gateway/src/proxy/splice.rs:294`

**Description**

Data fetched from outside the enclave is consumed in a decision that carries authority, with no signature check, attestation, or proof verification anywhere in this file. Whoever controls the transport, a parent-side proxy on Nitro, the network path on Confidential Space, chooses what that response says. Without end-to-end authentication the enclave is not deciding; it is relaying someone else's decision.

**Evidence**

```
let relay = async {
```

**Recommendations**

Require an end-to-end authenticated payload for every relayed response: a signature from the data source verified in-enclave against a pinned key, TLS terminated inside the enclave, or a chain state proof. Treat the KMS proxy as the blind transport it is documented to be, and verify the KMS response itself.

### [C-04] Data relayed by the parent consumed without verifying its signature

`dstack/http-client/src/lib.rs:75`

**Description**

Data fetched from outside the enclave is consumed in a decision that carries authority, with no signature check, attestation, or proof verification anywhere in this file. Whoever controls the transport, a parent-side proxy on Nitro, the network path on Confidential Space, chooses what that response says. Without end-to-end authentication the enclave is not deciding; it is relaying someone else's decision.

**Evidence**

```
let client = reqwest::Client::builder().build()?;
```

**Recommendations**

Require an end-to-end authenticated payload for every relayed response: a signature from the data source verified in-enclave against a pinned key, TLS terminated inside the enclave, or a chain state proof. Treat the KMS proxy as the blind transport it is documented to be, and verify the KMS response itself.

### [C-05] Attestation signature checked without validating the chain to the AWS Nitro root

`dstack/kms/src/main_service.rs:832`

**Description**

File assembles a certificate chain from the attestation document's cabundle and never verifies it against anything, the chain is built and discarded. This is the shape left behind when the verification call is removed and the plumbing around it survives. Verifying a document against the certificate embedded in that same document proves nothing: an attacker generates a keypair, signs a document asserting any PCR values they like, embeds their own certificate, and it verifies. The security of the scheme is entirely in walking the cabundle to an out-of-band pinned AWS Nitro root.

**Evidence**

```
attestation_doc: Vec::new(),
```

**Recommendations**

Validate the full certificate chain from the leaf through the supplied cabundle to a locally pinned copy of the AWS Nitro root certificate, check validity periods, and only then trust the PCR map. Prefer an audited implementation such as evervault/attestation-doc-validation or the AWS NSM API over hand-rolling COSE verification.

### [C-06] Data relayed by the parent consumed without verifying its signature

`dstack/nsm-qvl/src/collateral.rs:115`

**Description**

Data fetched from outside the enclave is consumed in a decision that carries authority, with no signature check, attestation, or proof verification anywhere in this file. Whoever controls the transport, a parent-side proxy on Nitro, the network path on Confidential Space, chooses what that response says. Without end-to-end authentication the enclave is not deciding; it is relaying someone else's decision.

**Evidence**

```
let response = reqwest::get(url)
```

**Recommendations**

Require an end-to-end authenticated payload for every relayed response: a signature from the data source verified in-enclave against a pinned key, TLS terminated inside the enclave, or a chain state proof. Treat the KMS proxy as the blind transport it is documented to be, and verify the KMS response itself.

## [C-07] Data relayed by the parent consumed without verifying its signature

`dstack/nvidia-attest-proxy/src/lib.rs:162`

**Description**

Data fetched from outside the enclave is consumed in a decision that carries authority, with no signature check, attestation, or proof verification anywhere in this file. Whoever controls the transport, a parent-side proxy on Nitro, the network path on Confidential Space, chooses what that response says. Without end-to-end authentication the enclave is not deciding; it is relaying someone else's decision.

**Evidence**

```
.redirect(reqwest::redirect::Policy::none())
```

**Recommendations**

Require an end-to-end authenticated payload for every relayed response: a signature from the data source verified in-enclave against a pinned key, TLS terminated inside the enclave, or a chain state proof. Treat the KMS proxy as the blind transport it is documented to be, and verify the KMS response itself.

## [C-08] Decrypted data or key material reaches a log sink outside the enclave

`dstack/scripts/bin/dstack-cloud:1050`

**Description**

Secret-derived value (f"Generated instance_id_seed: {app.instance_id_seed}") written to a log sink. Inside an enclave this reaches the operator, typically via CloudWatch. Log a keyed hash or truncated identifier.

**Evidence**

```
logger.info(f"Generated instance_id_seed: {app.instance_id_seed}")
```

**Recommendations**

Log hashed or truncated identifiers only. Where a secret-derived value must be correlated across systems, log a keyed hash of it. Route any genuinely sensitive diagnostic to an in-enclave buffer that is never flushed to the parent.

## [C-09] Decrypted data or key material reaches a log sink outside the enclave

`dstack/scripts/bin/dstack-cloud:1606`

**Description**

Secret-derived value (f"Generated instance_id_seed: {app.instance_id_seed}") written to a log sink. Inside an enclave this reaches the operator, typically via CloudWatch. Log a keyed hash or truncated identifier.

**Evidence**

```
logger.info(f"Generated instance_id_seed: {app.instance_id_seed}")
```

**Recommendations**

Log hashed or truncated identifiers only. Where a secret-derived value must be correlated across systems, log a keyed hash of it. Route any genuinely sensitive diagnostic to an in-enclave buffer that is never flushed to the parent.

### [C-10] Data relayed by the parent consumed without verifying its signature

`dstack/tpm-qvl/src/collateral.rs:168`

**Description**

Data fetched from outside the enclave is consumed in a decision that carries authority, with no signature check, attestation, or proof verification anywhere in this file. Whoever controls the transport, a parent-side proxy on Nitro, the network path on Confidential Space, chooses what that response says. Without end-to-end authentication the enclave is not deciding; it is relaying someone else's decision.

**Evidence**

```
reqwest::blocking::get(url).context(format!("failed to download CRL from {url}"))?;
```

**Recommendations**

Require an end-to-end authenticated payload for every relayed response: a signature from the data source verified in-enclave against a pinned key, TLS terminated inside the enclave, or a chain state proof. Treat the KMS proxy as the blind transport it is documented to be, and verify the KMS response itself.

### [C-11] Attestation signature checked without validating the chain to the AWS Nitro root

`dstack/verifier/src/verification.rs:1657`

**Description**

File assembles a certificate chain from the attestation document's cabundle and never verifies it against anything, the chain is built and discarded. This is the shape left behind when the verification call is removed and the plumbing around it survives. Verifying a document against the certificate embedded in that same document proves nothing: an attacker generates a keypair, signs a document asserting any PCR values they like, embeds their own certificate, and it verifies. The security of the scheme is entirely in walking the cabundle to an out-of-band pinned AWS Nitro root.

**Evidence**

```
attestation_doc: Vec::new(),
```

**Recommendations**

Validate the full certificate chain from the leaf through the supplied cabundle to a locally pinned copy of the AWS Nitro root certificate, check validity periods, and only then trust the PCR map. Prefer an audited implementation such as evervault/attestation-doc-validation or the AWS NSM API over hand-rolling COSE verification.

### [C-12] Data relayed by the parent consumed without verifying its signature

`dstack/vmm/src/main_service.rs:781`

#### Description

Data fetched from outside the enclave is consumed in a decision that carries authority, with no signature check, attestation, or proof verification anywhere in this file. Whoever controls the transport, a parent-side proxy on Nitro, the network path on Confidential Space, chooses what that response says. Without end-to-end authentication the enclave is not deciding; it is relaying someone else's decision.

#### Evidence

```
let kms = self.kms_client()?;
```

#### Recommendations

Require an end-to-end authenticated payload for every relayed response: a signature from the data source verified in-enclave against a pinned key, TLS terminated inside the enclave, or a chain state proof. Treat the KMS proxy as the blind transport it is documented to be, and verify the KMS response itself.

### [C-13] Certificate verification explicitly disabled on a trust-bearing connection

`sdk/go/ratls/ratls.go:246`

#### Description

Certificate verification disabled on a connection that leaves the enclave through the parent-side proxy.

#### Evidence

```
InsecureSkipVerify: true,
```

#### Recommendations

Never disable verification on a connection that leaves the enclave. Where the parent proxy terminates TLS, run an authenticated protocol inside the tunnel so the proxy stays a blind transport rather than a trusted one.

## 8.2. High findings

### [H-01] dstack deployment without KmsAuth or AppAuth code governance

`.:0`

#### Description

dstack's code-is-law property comes from two contracts: KmsAuth holds the global allowlist of authorised applications, AppAuth holds per-application authorised code hashes and upgrade rules. Enforcement happens at the KMS layer, so unauthorised code still executes but cannot obtain decryption keys and is functionally inert. Without registration the deployment looks like it works while the governance guarantee is simply absent.

#### Evidence

```
no KmsAuth/AppAuth reference found in repository
```

#### Recommendations

Register the application in KmsAuth, define authorised code hashes and upgrade logic in AppAuth, and confirm the KMS refuses key release for an unregistered hash before relying on the property.

### [H-02] Container or process output stream exposed across the enclave boundary by configuration

`dstack/crates/dstack-cli-core/src/compose.rs:61`

**Description**

Configuration publishes an output stream outright. Unlike a single leaked value, this exposes every future log statement inside the boundary, the defect is the channel, not the line, so no amount of care about what individual statements print will close it. Anything written to stdout or stderr from here should be treated as public.

**Evidence**

```
"public_logs": true,
```

**Recommendations**

Do not relay container or process output across the boundary. Where operators need diagnostics, emit a fixed vocabulary of event codes with no interpolated values, or require attestation-gated authentication on the log endpoint and document that anything written to it is public.

### [H-03] Container or process output stream exposed across the enclave boundary by configuration

`dstack/crates/dstack-cli-core/src/config.rs:238`

**Description**

Configuration publishes an output stream outright. Unlike a single leaked value, this exposes every future log statement inside the boundary, the defect is the channel, not the line, so no amount of care about what individual statements print will close it. Anything written to stdout or stderr from here should be treated as public.

**Evidence**

```
"public_logs": true,
```

**Recommendations**

Do not relay container or process output across the boundary. Where operators need diagnostics, emit a fixed vocabulary of event codes with no interpolated values, or require attestation-gated authentication on the log endpoint and document that anything written to it is public.

### [H-04] Host-supplied filesystem path used without rejecting symbolic links

`dstack/crates/dstack-volume/src/bin/dstack-volume.rs:390`

**Description**

Filesystem copy or read on a host-supplied path with no symlink rejection. std::fs and fs_err both dereference symbolic links, so an operator can substitute an expected input with a link to /proc/kcore or /proc/self/mem and have the enclave copy live kernel memory: which at early boot holds TDX injected randomness and ASLR offsets. Use symlink_metadata to reject links, and open with O_NOFOLLOW so the check cannot be raced.

**Evidence**

```
for line in fs::read_to_string("/proc/self/mountinfo")?.lines() {
```

**Recommendations**

Use symlink-aware metadata (symlink_metadata / lstat / O_NOFOLLOW) and reject a link before opening the target. Check-then-open is racy on its own, so open with O_NOFOLLOW or verify the opened descriptor, and treat the host-shared directory as hostile input rather than as a filesystem.

### [H-05] Container or process output stream exposed across the enclave boundary by configuration

`dstack/crates/dstackup/src/install.rs:259`

**Description**

An output stream is relayed out of the enclave with no authentication or attestation gate on the path. Unlike a single leaked value, this exposes every future log statement inside the boundary, the defect is the channel, not the line, so no amount of care about what individual statements print will close it. Anything written to stdout or stderr from here should be treated as public.

**Evidence**

```
println!("=> not ready within timeout (journalctl -u {vmm_unit})");
```

**Recommendations**

Do not relay container or process output across the boundary. Where operators need diagnostics, emit a fixed vocabulary of event codes with no interpolated values, or require attestation-gated authentication on the log endpoint and document that anything written to it is public.

### [H-06] Non-constant-time comparison of a secret-derived value

`dstack/dstack-mr/src/kernel.rs:30`

**Description**

Non-constant-time comparison on a secret-derived value (pe_sig). Comparison short-circuits on the first differing byte, so response latency leaks how many leading bytes matched and the value can be recovered byte by byte from the parent instance. Use subtle::ConstantTimeEq.

**Evidence**

```
if pe_sig != pe::IMAGE_NT_SIGNATURE {
```

**Recommendations**

Use subtle::ConstantTimeEq in Rust, hmac.compare_digest in Python, crypto.timingSafeEqual in Node, or crypto/subtle.ConstantTimeCompare in Go, for every comparison whose operands derive from a secret.

### [H-07] Host-supplied filesystem path used without rejecting symbolic links

`dstack/dstack-util/src/system_setup.rs:259`

**Description**

Filesystem copy or read on a host-supplied path with no symlink rejection. std::fs and fs_err both dereference symbolic links, so an operator can substitute an expected input with a link to /proc/kcore or /proc/self/mem and have the enclave copy live kernel memory: which at early boot holds TDX injected randomness and ASLR offsets. Use symlink_metadata to reject links, and open with O_NOFOLLOW so the check cannot be raced.

**Evidence**

```
let encrypted_env = fs::read(host_shared_dir.encrypted_env_file()).unwrap_or_default();
```

**Recommendations**

Use symlink-aware metadata (symlink_metadata / lstat / O_NOFOLLOW) and reject a link before opening the target. Check-then-open is racy on its own, so open with O_NOFOLLOW or verify the opened descriptor, and treat the host-shared directory as hostile input rather than as a filesystem.

### [H-08] Host-supplied filesystem path used without rejecting symbolic links

`dstack/dstack-util/src/system_setup.rs:970`

**Description**

Filesystem copy or read on a host-supplied path with no symlink rejection. std::fs and fs_err both dereference symbolic links, so an operator can substitute an expected input with a link to /proc/kcore or /proc/self/mem and have the enclave copy live kernel memory: which at early boot holds TDX injected randomness and ASLR offsets. Use symlink_metadata to reject links, and open with O_NOFOLLOW so the check cannot be raced.

**Evidence**

```
let user_config = fs::read_to_string(shared.dir.user_config_file())
```

**Recommendations**

Use symlink-aware metadata (symlink_metadata / lstat / O_NOFOLLOW) and reject a link before opening the target. Check-then-open is racy on its own, so open with O_NOFOLLOW or verify the opened descriptor, and treat the host-shared directory as hostile input rather than as a filesystem.

### [H-09] Non-constant-time comparison of a secret-derived value

`dstack/dstack-util/src/system_setup.rs:1125`

**Description**

Non-constant-time comparison on a secret-derived value (dstack_types::launch_token_hash(token)[..]). Comparison short-circuits on the first differing byte, so response latency leaks how many leading bytes matched and the value can be recovered byte by byte from the parent instance. Use subtle::ConstantTimeEq.

**Evidence**

```
if dstack_types::launch_token_hash(token)[..] != expected[..] {
```

**Recommendations**

Use subtle::ConstantTimeEq in Rust, hmac.compare_digest in Python, crypto.timingSafeEqual in Node, or crypto/subtle.ConstantTimeCompare in Go, for every comparison whose operands derive from a secret.

### [H-10] Container or process output stream exposed across the enclave boundary by configuration

`dstack/guest-agent/src/http_routes.rs:11`

**Description**

An output stream is relayed out of the enclave with no authentication or attestation gate on the path. Unlike a single leaked value, this exposes every future log statement inside the boundary, the defect is the channel, not the line, so no amount of care about what individual statements print will close it. Anything written to stdout or stderr from here should be treated as public.

**Evidence**

```
use docker_logs::parse_duration;
```

**Recommendations**

Do not relay container or process output across the boundary. Where operators need diagnostics, emit a fixed vocabulary of event codes with no interpolated values, or require attestation-gated authentication on

the log endpoint and document that anything written to it is public.

### [H-11] vsock message schema carries no nonce, counter, or signed timestamp

`dstack/http-client/src/hyper_vsock.rs:30`

**Description**

Messages are read off vsock with no nonce, counter, or signed timestamp in the schema at all. The parent sees every message the enclave receives and can send any of them again; a correctly signed message is still correctly signed the second time, so authentication alone does not prevent re-execution of a withdrawal or a state transition.

**Evidence**

```
vsock_stream: tokio_vsock::VsockStream,
```

**Recommendations**

Include a monotonic counter or single-use nonce inside the authenticated portion of every state-changing message, and track consumed values in enclave-side state that survives for the window in which replay is meaningful.

### [H-12] Blocking vsock read with no timeout

`dstack/http-client/src/hyper_vsock.rs:131`

**Description**

vsock listener or stream created without an explicit read/write timeout.

**Evidence**

```
VsockStream::connect(cid, port).await
```

**Recommendations**

Set explicit read, write, and idle timeouts on every vsock socket, bound the number of concurrent connections, and reap handlers that exceed a deadline.

### [H-13] vsock message schema carries no nonce, counter, or signed timestamp

`dstack/rocket-vsock-listener/src/lib.rs:36`

**Description**

Messages are read off vsock with no nonce, counter, or signed timestamp in the schema at all. The parent sees every message the enclave receives and can send any of them again; a correctly signed message is still correctly signed the second time, so authentication alone does not prevent re-execution of a withdrawal or a state transition.

**Evidence**

```
listener: vsock::VsockListener,
```

**Recommendations**

Include a monotonic counter or single-use nonce inside the authenticated portion of every state-changing message, and track consumed values in enclave-side state that survives for the window in which replay is meaningful.

### [H-14] Blocking vsock read with no timeout

`dstack/rocket-vsock-listener/src/lib.rs:264`

**Description**

vsock listener or stream created without an explicit read/write timeout.

**Evidence**

```
let result = VsockListener::bind(&endpoint);
```

**Recommendations**

Set explicit read, write, and idle timeouts on every vsock socket, bound the number of concurrent connections, and reap handlers that exceed a deadline.

### [H-15] Panic, crash dump, or backtrace path can emit enclave memory to the parent

`dstack/scripts/bin/dstack-cloud:3668`

**Description**

Traceback printed or formatted inside an enclave. Tracebacks can carry local variable content across the trust boundary. Emit a static error code instead.

**Evidence**

```
traceback.print_exc()
```

**Recommendations**

Set panic=abort and disable core dumps in the enclave image. Install a panic hook that emits a static error code and nothing else. Wrap secrets in types whose Debug implementation is redacted, so a backtrace cannot print them even by accident.

### [H-16] vsock message schema carries no nonce, counter, or signed timestamp

`dstack/tdx-attest/src/linux.rs:582`

**Description**

Messages are read off vsock with no nonce, counter, or signed timestamp in the schema at all. The parent sees every message the enclave receives and can send any of them again; a correctly signed message is still correctly signed the second time, so authentication alone does not prevent re-execution of a withdrawal or a state transition.

**Evidence**

```
let mut stream = vsock::VsockStream::connect_with_cid_port(vsock::VMADDR_CID_HOST, vsock_por t)?;
```

**Recommendations**

Include a monotonic counter or single-use nonce inside the authenticated portion of every state-changing message, and track consumed values in enclave-side state that survives for the window in which replay is meaningful.

### [H-17] vsock message schema carries no nonce, counter, or signed timestamp

`dstack/vmm/src/main.rs:154`

**Description**

Messages are read off vsock with no nonce, counter, or signed timestamp in the schema at all. The parent sees every message the enclave receives and can send any of them again; a correctly signed message is still

correctly signed the second time, so authentication alone does not prevent re-execution of a withdrawal or a state transition.

**Evidence**

```
let listener = VsockListener::bind_rocket(&ignite)
```

**Recommendations**

Include a monotonic counter or single-use nonce inside the authenticated portion of every state-changing message, and track consumed values in enclave-side state that survives for the window in which replay is meaningful.

### [H-18] Non-constant-time comparison of a secret-derived value

`dstack/vmm/src/vmm-cli.py:985`

**Description**

Non-constant-time comparison on a secret-derived value (app_compose.get("launch_token_hash")). Use hmac.compare_digest; == on bytes or str short-circuits and leaks the matching prefix length through response timing.

**Evidence**

```
if app_compose.get("launch_token_hash") != launch_token_hash:
```

**Recommendations**

Use subtle::ConstantTimeEq in Rust, hmac.compare_digest in Python, crypto.timingSafeEqual in Node, or crypto/subtle.ConstantTimeCompare in Go, for every comparison whose operands derive from a secret.

### [H-19] Non-constant-time comparison of a secret-derived value

`dstack/vmm/src/vmm-cli.py:1148`

**Description**

Non-constant-time comparison on a secret-derived value (app_compose.get("launch_token_hash")). Use hmac.compare_digest; == on bytes or str short-circuits and leaks the matching prefix length through response timing.

**Evidence**

```
if app_compose.get("launch_token_hash") != launch_token_hash:
```

**Recommendations**

Use subtle::ConstantTimeEq in Rust, hmac.compare_digest in Python, crypto.timingSafeEqual in Node, or crypto/subtle.ConstantTimeCompare in Go, for every comparison whose operands derive from a secret.

### [H-20] Container or process output stream exposed across the enclave boundary by configuration

`dstack/vmm/ui/src/composables/useVmManager.ts:229`

**Description**

Configuration publishes an output stream outright. Unlike a single leaked value, this exposes every future log statement inside the boundary, the defect is the channel, not the line, so no amount of care about what individual statements print will close it. Anything written to stdout or stderr from here should be treated as public.

**Evidence**

```
public_logs: true,
```

**Recommendations**

Do not relay container or process output across the boundary. Where operators need diagnostics, emit a fixed vocabulary of event codes with no interpolated values, or require attestation-gated authentication on the log endpoint and document that anything written to it is public.

### [H-21] Container or process output stream exposed across the enclave boundary by configuration

`tools/sca/sca.py:62`

**Description**

Configuration publishes an output stream outright. Unlike a single leaked value, this exposes every future log statement inside the boundary, the defect is the channel, not the line, so no amount of care about what individual statements print will close it. Anything written to stdout or stderr from here should be treated as public.

**Evidence**

```
"public_logs": True,
```

**Recommendations**

Do not relay container or process output across the boundary. Where operators need diagnostics, emit a fixed vocabulary of event codes with no interpolated values, or require attestation-gated authentication on the log endpoint and document that anything written to it is public.

## 8.3. Medium findings

### [M-01] Keys sealed to hardware rather than derived from application identity

`dstack/tpm-attest/src/esapi.rs:109`

**Description**

Key material appears sealed to hardware with no application-identity derivation alongside it. dstack derives root keys using the application identifier as a KDF parameter so a workload can migrate across physical TEEs and vendors; hardware sealing pins the key to one machine and removes migration as a response to compromise, which dstack's own threat model assumes will eventually happen.

**Evidence**

```
.seal(data, parent_handle, &tpm_selection, hash_alg)
```

**Recommendations**

Derive keys through dstack-KMS using the application identifier so that identity follows the workload rather than the machine.

### [M-02] Keys sealed to hardware rather than derived from application identity

`dstack/tpm-attest/src/lib.rs:213`

**Description**

Key material appears sealed to hardware with no application-identity derivation alongside it. dstack derives root keys using the application identifier as a KDF parameter so a workload can migrate across physical TEEs and vendors; hardware sealing pins the key to one machine and removes migration as a response to compromise, which dstack's own threat model assumes will eventually happen.

**Evidence**

```
let (pub_bytes, priv_bytes) = ctx.seal(data, parent_handle, pcr_selection)?;
```

**Recommendations**

Derive keys through dstack-KMS using the application identifier so that identity follows the workload rather than the machine.

### [M-03] Keys sealed to hardware rather than derived from application identity

`dstack/tpm2/src/bin/tpm2-test.rs:45`

**Description**

Key material appears sealed to hardware with no application-identity derivation alongside it. dstack derives root keys using the application identifier as a KDF parameter so a workload can migrate across physical TEEs and vendors; hardware sealing pins the key to one machine and removes migration as a response to compromise, which dstack's own threat model assumes will eventually happen.

**Evidence**

```
"seal" => test_seal_unseal(),
```

**Recommendations**

Derive keys through dstack-KMS using the application identifier so that identity follows the workload rather than the machine.

### [M-04] Keys sealed to hardware rather than derived from application identity

`dstack/tpm2/src/commands.rs:732`

**Description**

Key material appears sealed to hardware with no application-identity derivation alongside it. dstack derives root keys using the application identifier as a KDF parameter so a workload can migrate across physical TEEs and vendors; hardware sealing pins the key to one machine and removes migration as a response to compromise, which dstack's own threat model assumes will eventually happen.

**Evidence**

```
response.ensure_success().context("Create (seal) failed")?;
```

**Recommendations**

Derive keys through dstack-KMS using the application identifier so that identity follows the workload rather than the machine.

### [M-05] Keys sealed to hardware rather than derived from application identity

`dstack/tpm2/src/constants.rs:22`

**Description**

Key material appears sealed to hardware with no application-identity derivation alongside it. dstack derives root keys using the application identifier as a KDF parameter so a workload can migrate across physical

TEEs and vendors; hardware sealing pins the key to one machine and removes migration as a response to compromise, which dstack's own threat model assumes will eventually happen.

**Evidence**

```
Unseal = 0x0000015E,
```

**Recommendations**

Derive keys through dstack-KMS using the application identifier so that identity follows the workload rather than the machine.

### [M-06] Firmware recipe defines a secure-boot option and leaves it off

`os/yocto/layers/meta-dstack/recipes-core/dstack-ovmf/dstack-ovmf_git.bb:12`

**Description**

The firmware recipe defines a secure-boot option and leaves it out of the default configuration. Without it the firmware will load an unsigned bootloader, so the measured-boot chain starts from something the host can replace, which is the assumption every later measurement depends on. Enable it in PACKAGECONFIG, or state in the deployment documentation which layer is expected to provide the equivalent.

**Evidence**

```
PACKAGECONFIG ??= ""
```

**Recommendations**

Add secureboot to PACKAGECONFIG, or document which layer of the deployment provides the equivalent guarantee and how that is verified.

## 8.4. Low findings

### [L-01] Enclave image build is not reproducible, so PCR values cannot be independently confirmed

`tools/mock-cf-dns-api/Dockerfile:11`

**Description**

Package install without pinned versions (requirements.txt). Two builds of the same source will measure differently, which breaks independent verification of PCR values.

**Evidence**

```
RUN pip install --no-cache-dir -r requirements.txt
```

**Recommendations**

Pin base images by digest rather than tag, pin every package version, set SOURCE_DATE_EPOCH, and publish the expected PCR values alongside each release so anyone can verify the build.